Lab 4 Sensing and Control with Arduino

Drafted by Yuemin Ding

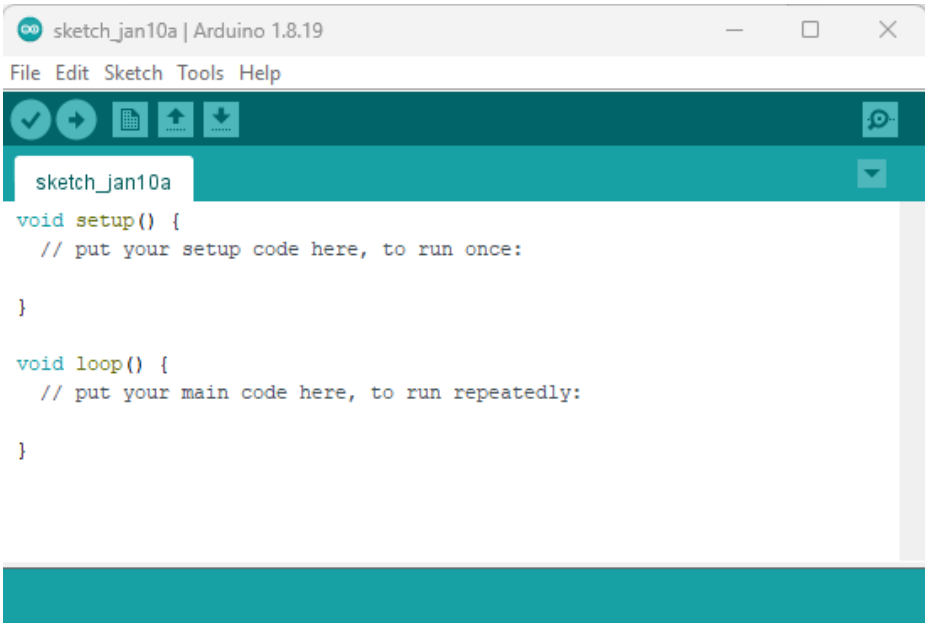
(If you are familiar with the Arduino ecosystem, you may directly work on the practice task at the end of the file.)

1. Environment configuration

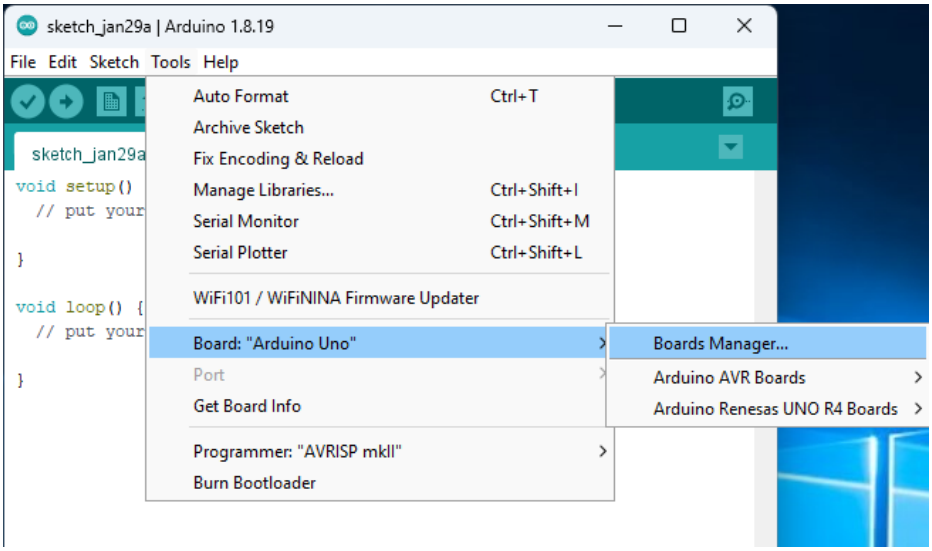
Step 1: Download and install Arduino IDE, available here: https://www.arduino.cc/en/software. (There are different versions, this tutorial uses 1.8.19 but it is OK to the latest version.)

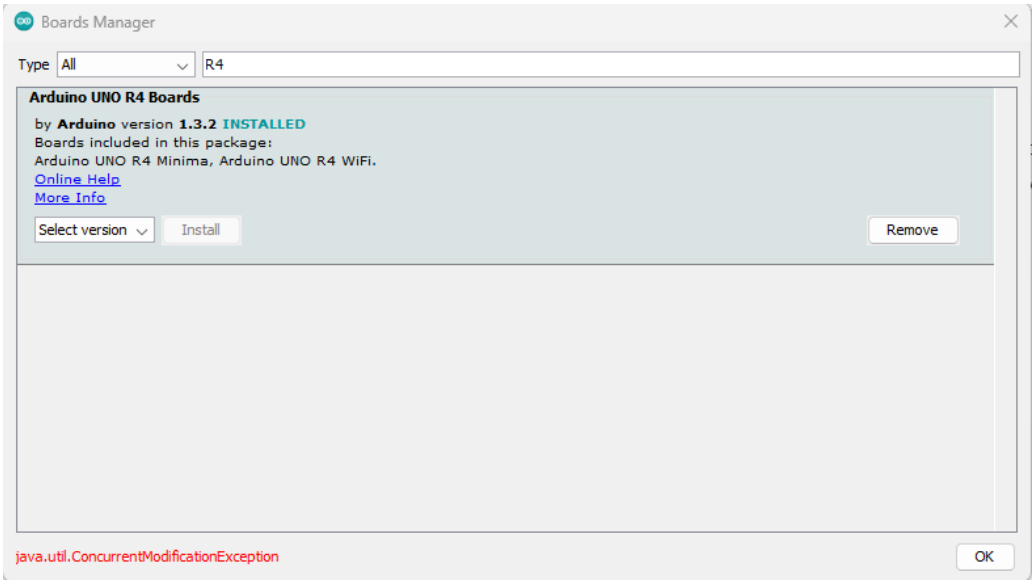
Step 2: Connect Arduino UNO R4 to PC using USB cable. Wait until the driver is installed and ready to use.

Step 3: Open Arduino IDE application software. By default, it comes out a minimum sketch with only setup() and loop() functions.

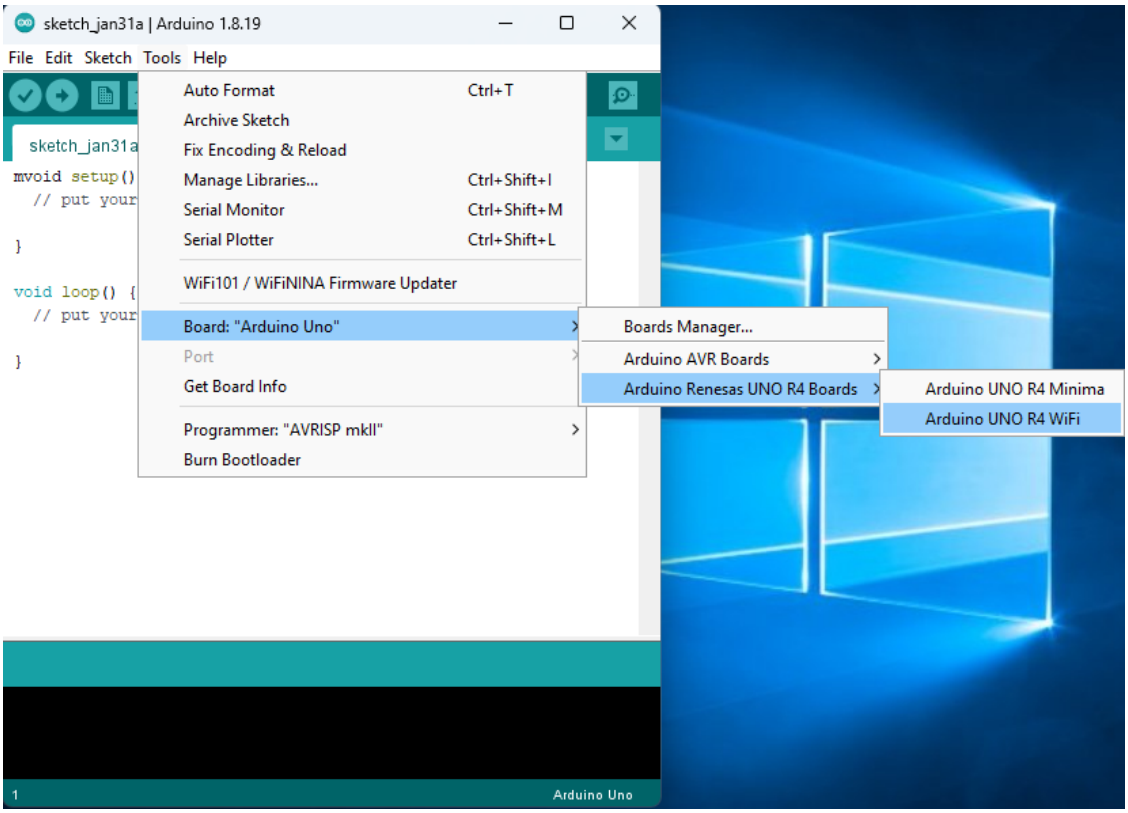


Step 4: Before start writing code, it is necessary first to configure the Arduino IDE. Our Arduino board is UNO R4 WiFi, which has not been integrated to the Arduino IDE by default. You may add it from "Tools-> Board->Boards Manager…". In the pop-up dialog, search for R4 and install the board library.

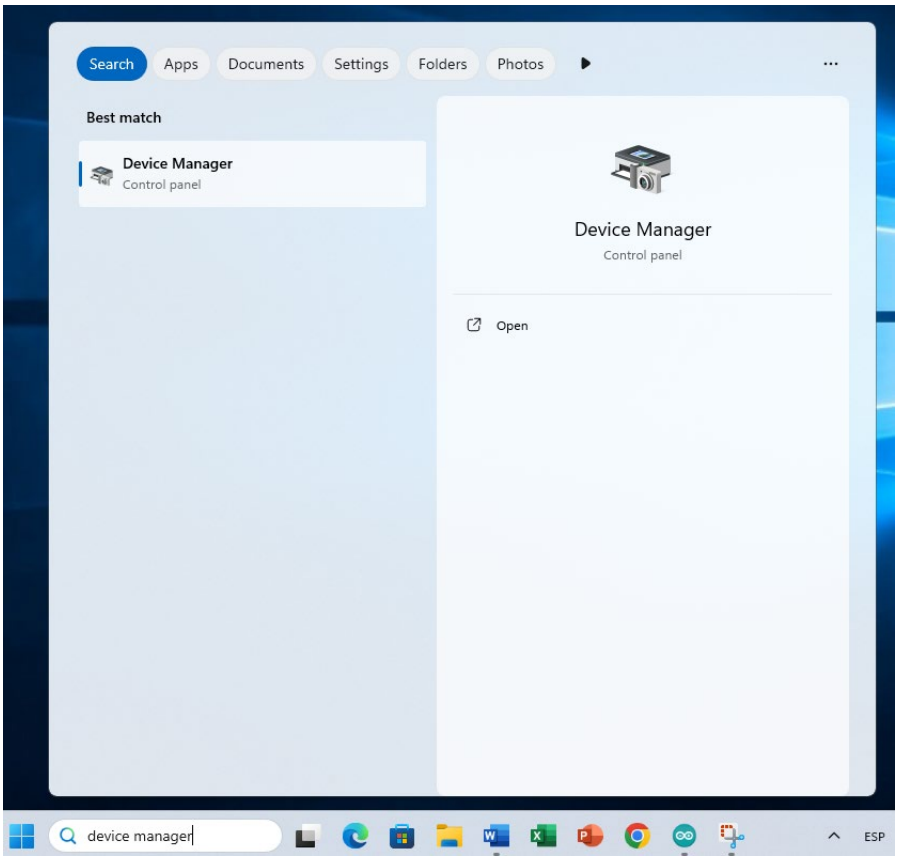




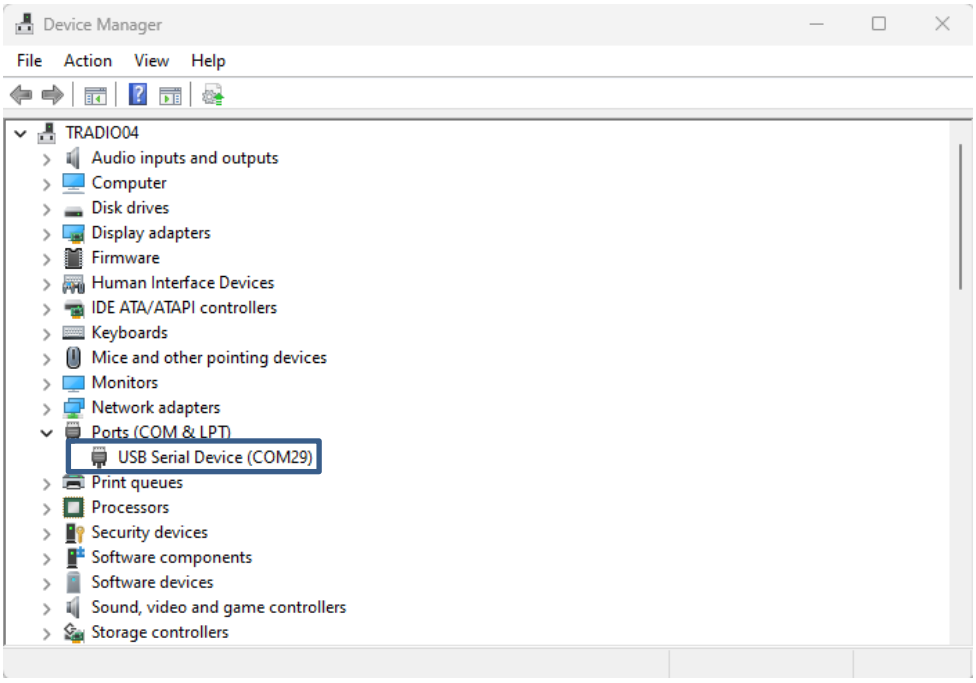
Step 5: After installing the board library, we still need to explicitly set the board in Arduino IDE. For this, go to "Tools->Board->Arduino Renesas UNO R4 Boards->Arduino UNO R4 Wifi".

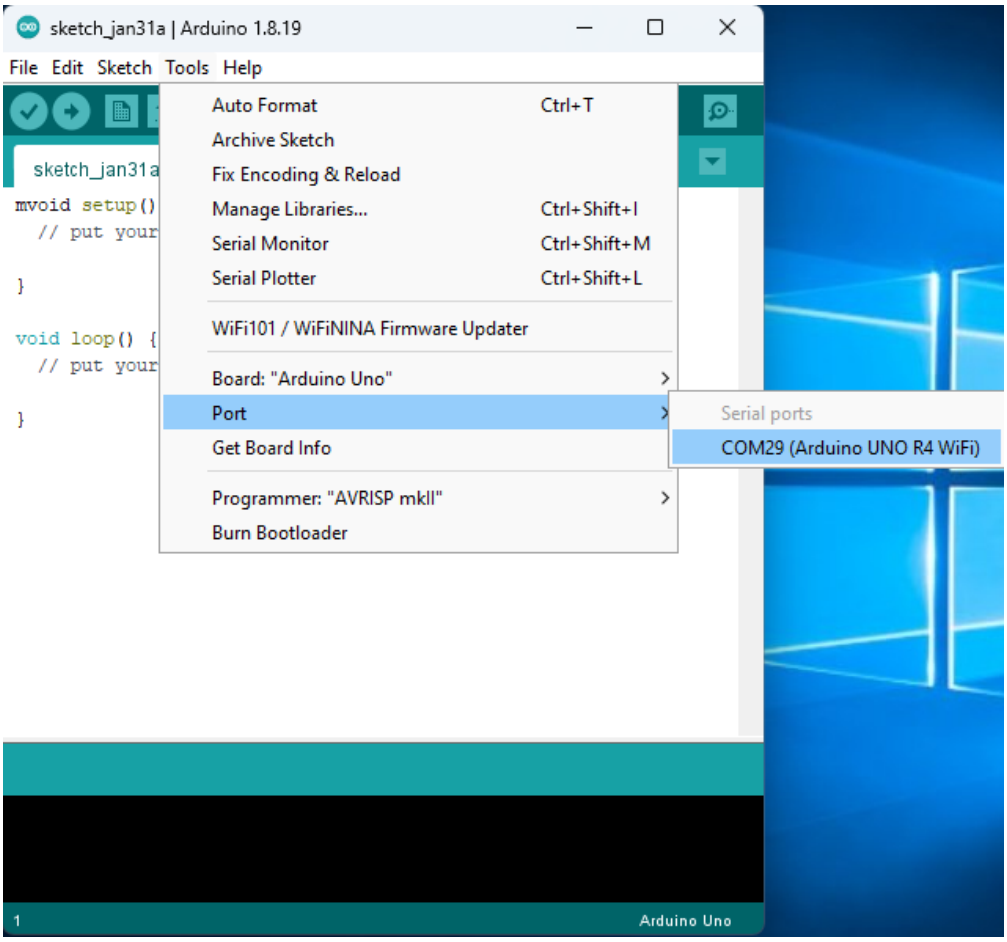


Step 6: The other important thing is to set the 'Port' number. It tells the computer which Serial Interface the Arduino board is connected to. To do so, open the ´Device Manager' of the computer.



The Device Manager indicates the hardware devices connected to the computer. Go to the ´Ports´ section, which lists all the Serial Communication devices. If you are not sure which one is for Arduino UNO, unplug it and see which one disappears (the number may change on different computers). Go back to Arduino IDE and set the port number accordingly.





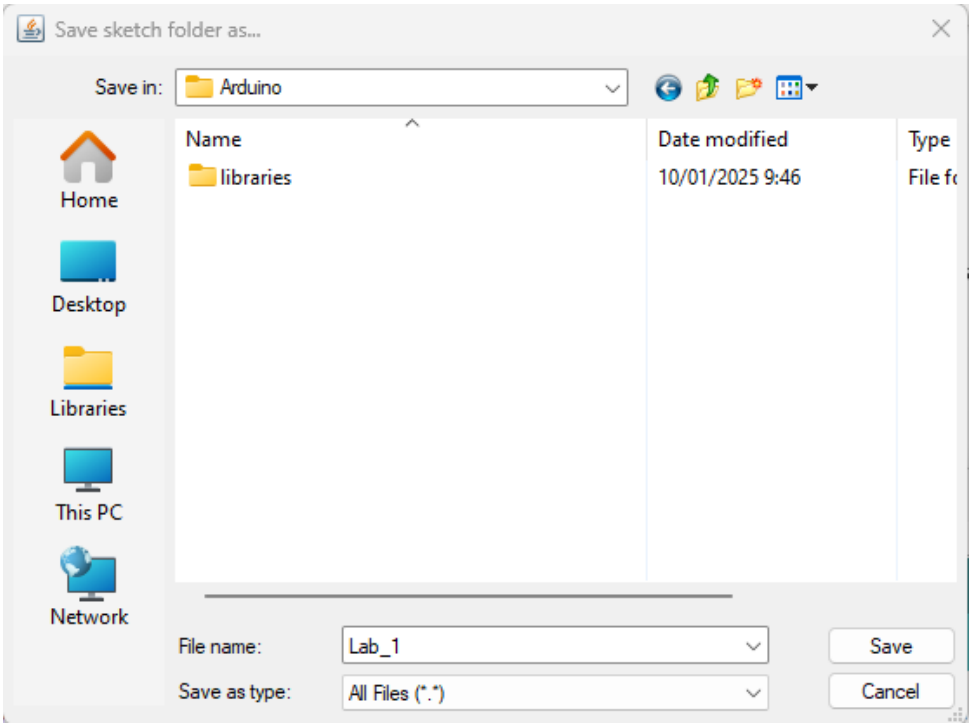
2. Communication with PC

Step 1: After configuring the Arduino IDE, we are ready to write our C/C++ program to send a message to PC. Refer to the following code.

```
void setup() {
  // put your setup code here, to run once:
 Serial.begin(9600);  //set the baudrate to 9600 bps
 Serial.println("Hello, this the Arduino UNO R3!");
 Serial.println("Type the mathemical constant that you would like to know,
select from (e, pi)");
}

void loop() {
  // put your main code here, to run repeatedly:
 while(Serial.available()){
  String para = Serial.readStringUntil('\n');
  para.trim(); //remove potential space symbols
  checkPara(para);
 }
}
```

```
void checkPara(String para){
  double x = 3.1415;
  double e = 2.7182;

  if (para == "e"){
    Serial.print("The value of 'e' is ");
    Serial.println(e);
    }
  else if(para == "pi"){
    Serial.print("The value of 'pi' is ");
    Serial.println(x);
    }
  else{
    Serial.println("Sorry, I do not know the value of '" + para + "'");
    }
}
```

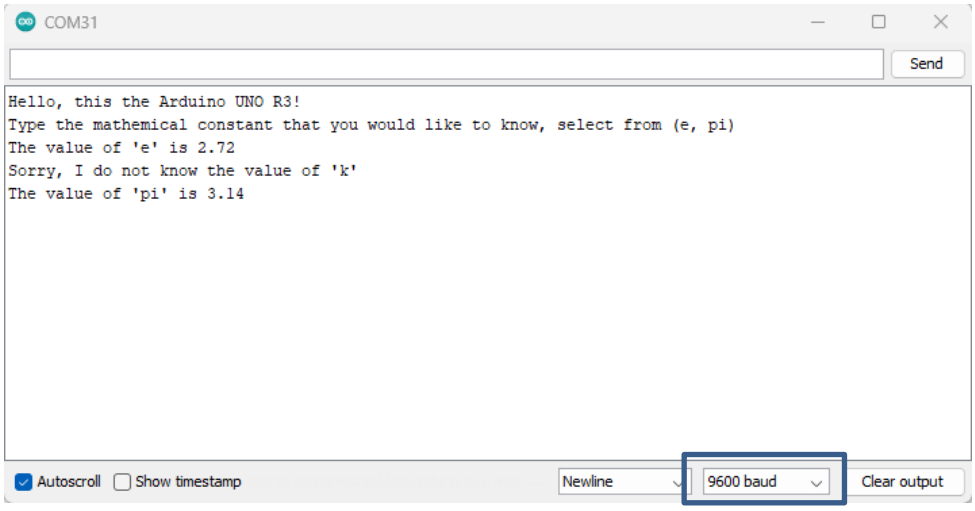
Step 2: Save the code from File->Save. Type the file name, select a directory that you prefer.



Step 3: Click the button to compile the code and verify if there is any error. Fix the errors if there are any until the Arduino IDE returns 'Done compiling'.

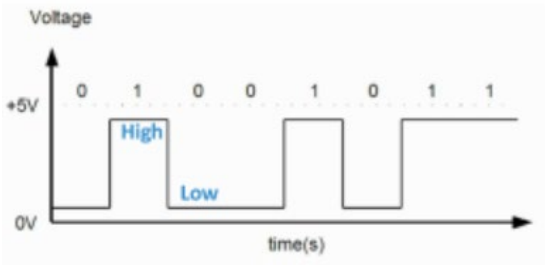
Step 4: Click the button to upload the compiled binary file to the Arduino UNO R4 board. It may take a while to upload. Wait until the IDE returns ´Done uploading´.

Step 5: Click the button to open the Serial Monitor. The messages we asked the Arduino Board to send to the PC were shown. If there is no message coming or random symbols are received, check whether the baud rate is set the same as indicated in the program. You may ask the Arduino Board to send other messages to the PC. Try as you like.



3. Read/write digital signals

Digital signals are commonly used to generate high/low voltage signals, such as control an LED or read a button, as shown below.



To do so, let us connect an LED module to Arduino UNO. Note: Before using Arduino pins for read/write, it is necessary to first set the pin mode to INPUT/OUTPUT.

LED pins	Arduino pins
´-´	GND
´+´	Pin2

Try the following code to see if the LED blinks.

```
void setup()
{
  pinMode(2, OUTPUT); // set the pin mode to output to write signal
}
void loop()
{
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);
    delay(1000);
}
```

Here, delay() function is used but it blocks the program. It means that during that second, Arduino is doing

nothing but just waiting. Another function counts time without blocking the program is millis(), which is normally used to trigger multiple events in Arduino. Here is an example code:

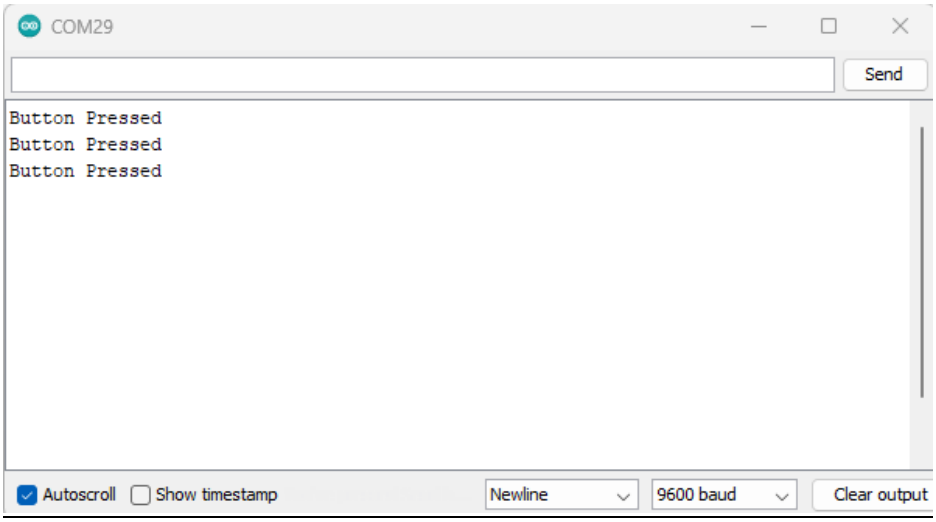
```
unsigned long previousMillis = 0;  // Stores the last time the LED state
was changed
const long interval = 1000;         // 1 second interval in milliseconds
int ledState = LOW;                 // Initial state of the LED (LOW means
off)

void setup() {
  pinMode(2, OUTPUT);  // Set pin 2 as output
}

void loop() {
  unsigned long currentMillis = millis();  // Get the current time

  // Check if 1 second has passed
  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;  // Save the current time for the next
interval

    // Toggle the LED state (HIGH <-> LOW)
    if (ledState == LOW) {
      ledState = HIGH;  // Change state to HIGH
    } else {
      ledState = LOW;   // Change state to LOW
    }

    digitalWrite(2, ledState);  // Set the LED to the current state
  }
}
```

We may also use Arduino to monitor if a button is pressed or not. If the button is pressed, it sends a serial message to the PC. Referring to the following hardware connection and the software code below.

Button module pins	Arduino pins
´-´	GND
´+´	5V
S	Pin2

```
const int buttonPin = 2;  // Pin connected to the button
int buttonState = 0;       // Variable to store the button state

void setup() {
```

```
  pinMode(buttonPin, INPUT);  // Set the button pin as input with the
internal pull-up resistor enabled
  Serial.begin(9600);  // Start serial communication to print the button
state
}

void loop() {
  buttonState = digitalRead(buttonPin);  // Read the state of the button

  if (buttonState == LOW) {
    // Button is pressed (since the button pulls the pin LOW when pressed)
    Serial.println("Button Pressed");
  }
  delay(200);  // Small delay for debouncing (optional)
}
```

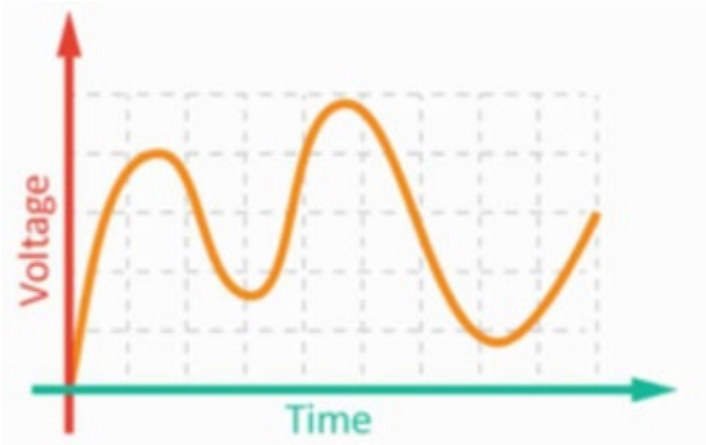


4. Read/write analog signals

Many sensors generate Analog Signals, see below. Analog signals are generally bound to a voltage range. In an Arduino, that voltage range is typically 0-5V, or 0-3.3V. The Arduino UNO has a 10-bit ADC, for which the minimum value is 0 and the maximum value is 5.

The analog voltage is then digitalized through a 10-bit ADC (analog-to-digital converter) into digital values, e.g. '0's and '1's. The mapping is linear with 0V -> '0000000000' (0 in decimal) and 5V ->'1111111111' (1023 in decimal).

The function, '**analogRead(pin)**', is used to get the digital value of the voltage signal, such as 'analogRead(A0)'. With this function, we will obtain a 10-bit binary value, e.g. '0101010101' corresponding to a decimal value of 682. The voltage can be obtained using 682/1023 * 5V = 3.33 V.

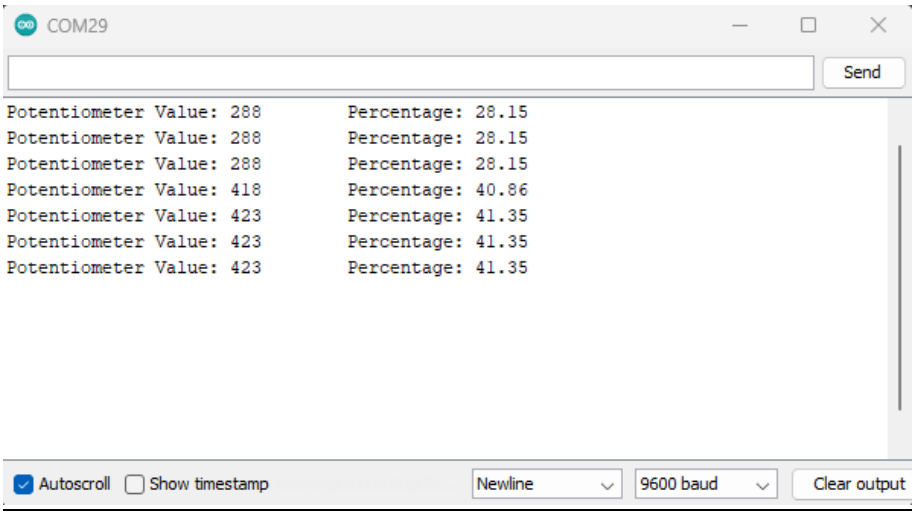


Here, this lab reads the output from a Potentiometer, which is commonly used to sense the opening of valves. The ADC is used to convert the analog voltage to digital values. Both the digital value and the percentage values are output to the serial interface. Here is the hardware connection:

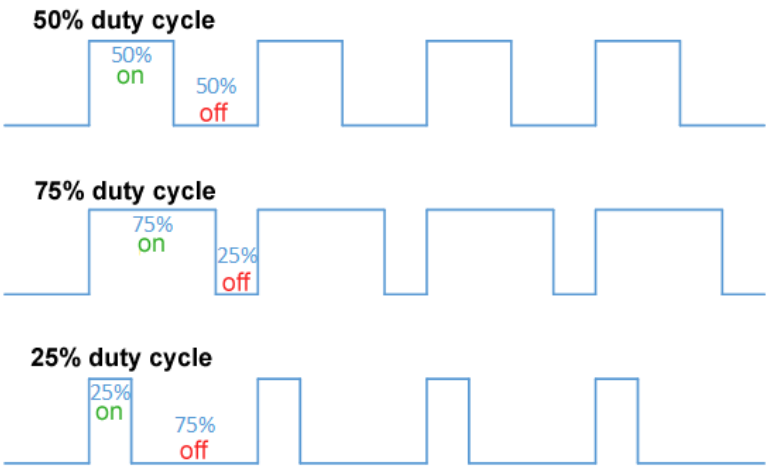
Potentiometer pins	Arduino pins
´-´	GND
´+´	5V
A	A0

```
const int potPin = A0;  // Analog pin connected to the potentiometer
int potValue = 0;       // Variable to store the potentiometer value
float potPercentage = 0.0; // Variable to store the potentiometer percentage

void setup() {
  Serial.begin(9600);  // Start serial communication for monitoring the
potentiometer value
}

void loop() {
  potValue = analogRead(potPin);  // Read the analog value from the
potentiometer

  // Convert the analog value to a percentage
  potPercentage = (potValue / 1023.0) * 100.0;  // Formula to convert to
percentage

  Serial.print("Potentiometer Value: ");
  Serial.print(potValue);  // Print the analog value (0-1023)
  Serial.print("\tPercentage: ");
  Serial.println(potPercentage);  // Print the potentiometer value as a
percentage (0-100)
```

```
  delay(1000);  // Delay for a short period to avoid flooding the serial
output
}
```



The other common function is to write analog signals for control purposes with Pulse-Width Modulation (PWM), see below. The duty cycle represents the strength of the analog output. For instance, if the maximum output voltage is 12V, a 50% duty cycle gives 6V. **Note: only the Arduino pins marked by '~' symbols support writing analog signals.**



Here, we use analog signals to control an RGB LED. An RGB LED has three channels, namely 'red', 'green', and 'blue'. The brightness of each channel can be controlled with analog signals. The mix of these three channels allows the RGB LED to show different colors.

RGB pins	Arduino pins
+	5V
R	9
G	10
B	11

```
const int redPin = 9;
const int greenPin = 10;
const int bluePin = 11;

void setup() {
  // Set the RGB pins as outputs
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
}

void loop() {
  // Create colors by setting the appropriate PWM values

  // Red color
  analogWrite(redPin, 255);   // Full brightness for red
  analogWrite(greenPin, 0);   // No green
  analogWrite(bluePin, 0);    // No blue
  delay(1000);  // Wait for 1 second

  // Green color
  analogWrite(redPin, 0);    // No red
  analogWrite(greenPin, 255);  // Full brightness for green
  analogWrite(bluePin, 0);   // No blue
  delay(1000);  // Wait for 1 second

  // Blue color
  analogWrite(redPin, 0);    // No red
  analogWrite(greenPin, 0);  // No green
  analogWrite(bluePin, 255);   // Full brightness for blue
  delay(1000);  // Wait for 1 second

  // Purple color (Red + Blue)
  analogWrite(redPin, 255);   // Full brightness for red
  analogWrite(greenPin, 0);   // No green
  analogWrite(bluePin, 255);   // Full brightness for blue
  delay(1000);  // Wait for 1 second

  // Yellow color (Red + Green)
  analogWrite(redPin, 255);   // Full brightness for red
  analogWrite(greenPin, 255);  // Full brightness for green
  analogWrite(bluePin, 0);    // No blue
  delay(1000);  // Wait for 1 second
```

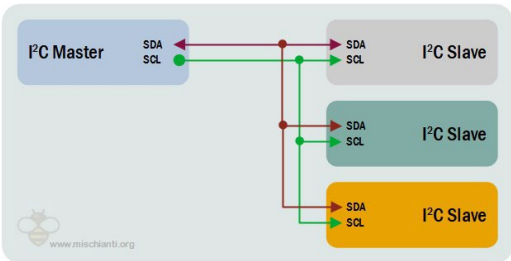
```
  // Cyan color (Green + Blue)
  analogWrite(redPin, 0);     // No red
  analogWrite(greenPin, 255);  // Full brightness for green
  analogWrite(bluePin, 255);   // Full brightness for blue
  delay(1000);  // Wait for 1 second

  // White color (All on)
  analogWrite(redPin, 255);   // Full brightness for red
  analogWrite(greenPin, 255);  // Full brightness for green
  analogWrite(bluePin, 255);   // Full brightness for blue
  delay(1000);  // Wait for 1 second
}
```

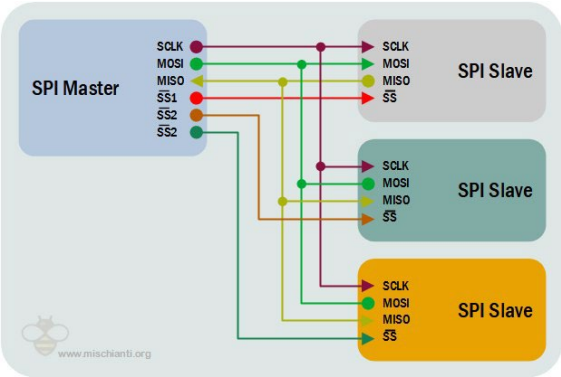
5. Sensor modules with serial communication interface

Many sensors give neither "HIGH/LOW" signals, e. g. button, nor analog signals, e. g. Potentiometer. These use Serial Communication to interact with the Arduino board. In other words, we need to read the sensing values from the sensor module following a specific protocol. Typical serial communication protocols include I2C, UART, and SPI. (Refer here for a detailed explanation: https://mischianti.org/i2c-spi-and-uart-compared/). In short, UART supports one-to-one communication, SPI supports one-to-one and one-to-many communications, and I2C supports one-to-one, one-to-many, and many-to-many communications. Implementing the protocol to interact with these sensors is not so easy. However, the manufacturer of these sensor modules normally offers an Arduino library and example codes for easy use of their products.

For each serial interface, the signal line connections are given below. Note that, in addition to the signal lines, the power and ground lines still need to be connected. These serial communication protocols are commonly supported by Arduino ecosystem. Refer below for more information:

- UART library → https://docs.arduino.cc/language-reference/en/functions/communication/serial/
- Software UART library → https://docs.arduino.cc/learn/built-in-libraries/software-serial/
- SPI library → https://docs.arduino.cc/language-reference/en/functions/communication/SPI/
- I2C library → https://docs.arduino.cc/language-reference/en/functions/communication/wire/

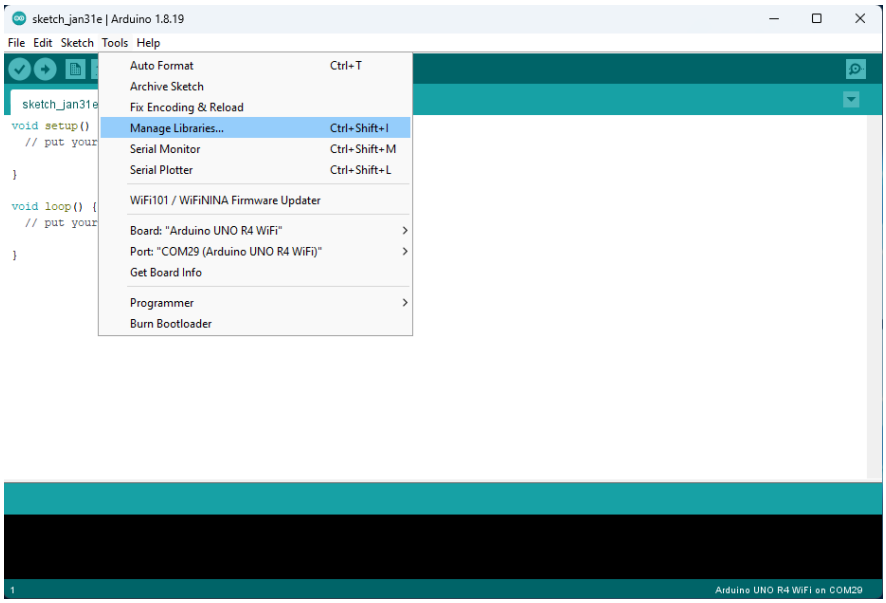


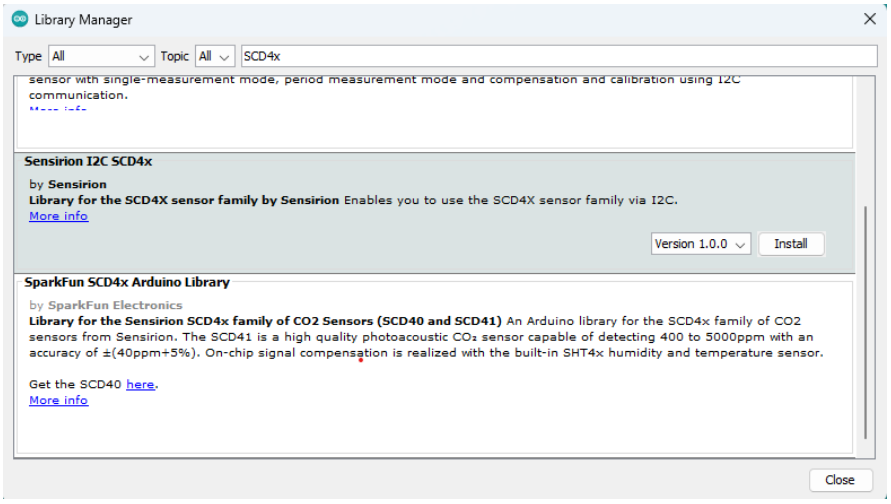


Here, we have a Sensirion industry-level CO2/temperature/humidity sensor, SCD4x with an I2C interface. To work with this sensor, let us first connect the hardware, referring to below:

SCD4x pins	Arduino pins
SCL	19/SCL
GND	GND
VDD	3.3V
SDA	18/SDA

After that, install the library from “Tools->Manage Libraries…”. In the pop-up dialog, search for ‘SCD4x’ and look for the official one from Sensirion. Install the library and try the code below.

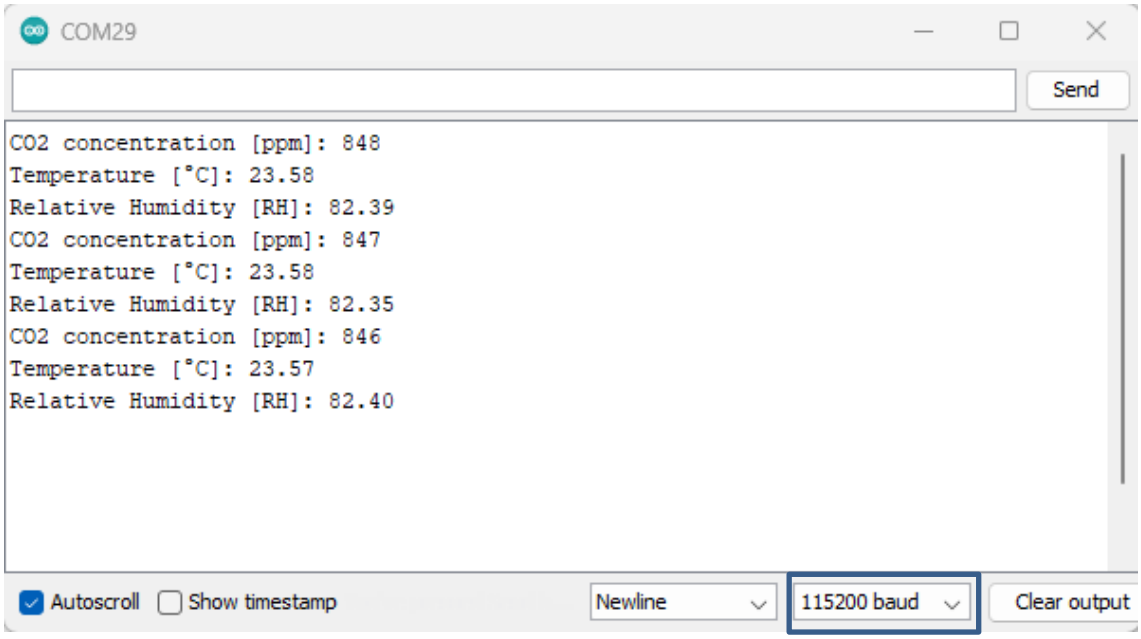





```
#include <Arduino.h>
#include <SensirionI2cScd4x.h>
#include <Wire.h>

// macro definitions
// make sure that we use the proper definition of NO_ERROR
#ifdef NO_ERROR
#undef NO_ERROR
#endif
#define NO_ERROR 0

SensirionI2cScd4x sensor;

static char errorMessage[64];
static int16_t error;

void PrintUint64(uint64_t& value) {
    Serial.print("0x");
    Serial.print((uint32_t)(value >> 32), HEX);
    Serial.print((uint32_t)(value & 0xFFFFFFFF), HEX);
}

void setup() {

    Serial.begin(115200);
    while (!Serial) {
        delay(100);
    }
    Wire.begin();
    sensor.begin(Wire, SCD41_I2C_ADDR_62);
```

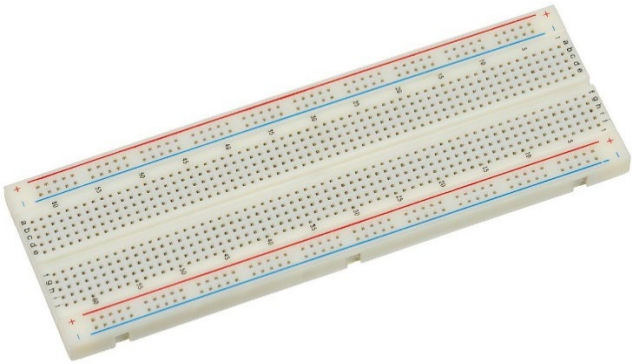
```
    uint64_t serialNumber = 0;
    delay(30);
    // Ensure sensor is in clean state
    error = sensor.wakeUp();
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute wakeUp(): ");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
    }
    error = sensor.stopPeriodicMeasurement();
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute stopPeriodicMeasurement(): 
");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
    }
    error = sensor.reinit();
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute reinit(): ");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
    }
    // Read out information about the sensor
    error = sensor.getSerialNumber(serialNumber);
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute getSerialNumber(): ");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
        return;
    }
    Serial.print("serial number: 0x");
    PrintUint64(serialNumber);
    Serial.println();
}

void loop() {

    uint16_t co2Concentration = 0;
    float temperature = 0.0;
    float relativeHumidity = 0.0;
    //
    // Wake the sensor up from sleep mode.
    //
    error = sensor.wakeUp();
```

```
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute wakeUp(): ");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
        return;
    }
    //
    // Ignore first measurement after wake up.
    //
    error = sensor.measureSingleShot();
    if (error != NO_ERROR) {
        Serial.print("Error trying to execute measureSingleShot(): ");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
        return;
    }
    //
    // Perform single shot measurement and read data.
    //
    error = sensor.measureAndReadSingleShot(co2Concentration,
temperature,relativeHumidity);

    if (error != NO_ERROR) {
        Serial.print("Error trying to execute measureAndReadSingleShot():
");
        errorToString(error, errorMessage, sizeof errorMessage);
        Serial.println(errorMessage);
        return;
    }
    //
    // Print results in physical units.
    //
    Serial.print("CO2 concentration [ppm]: ");
    Serial.print(co2Concentration);
    Serial.println();
    Serial.print("Temperature [°C]: ");
    Serial.print(temperature);
    Serial.println();
    Serial.print("Relative Humidity [RH]: ");
    Serial.print(relativeHumidity);
    Serial.println();
    delay(20000);
}
```

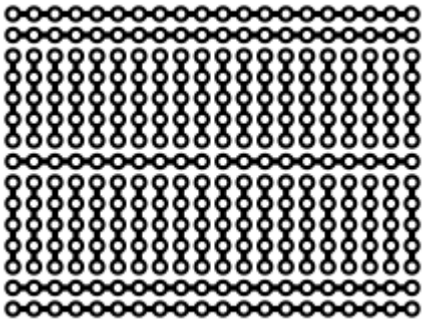


6. Solderless Breadboard

When connecting multiple sensors, a solderless breadboard would be helpful with (a) showing the appearance and (b) the underlining connections. The left-most and right-most two columns are connected separately for power supply and ground. The columns mark by '+' (red line) are used for power supply and those marked by '-' (blue line) are used for ground. **Caution: Do not connect them together to avoid short circuit, which may break the Arduino board**.



(a) Solderless breadboard



(b) Internal connections

Practice task:

Design an Arduino code to implement the following functions:

- Read the value of the Potentiometer every 0.5s and control the RGB LED, (0-40% → Green; 40%-80%V→Yellow, and 80%-100%V → Red).
- When the value is between 80%-100%, blink the another LED every 1 second. Otherwise, set it in an OFF state.
- Document the process to be submitted together with the code.